

ROUA TRADING

EXECUTIVE REPAIR PLAN

الخطة التنفيذية للإصلاح الشامل

إصلاح حرج - تصحيح بيانات - تحسين خوارزمي - تطوير هيكل

وثيقة تنفيذية شاملة لمعالجة جميع الأخطاء الحرجة والشذوذ في البيانات
والمشاكل الخوارزمية المكتشفة في منصة Roua Trading، مع جدول
زمني محدد وأولويات مرتبة حسب الأثر المالي والفني

السرية: عالي

الإصدار: 1.0

التاريخ: يونيو 2026

فهرس المحتويات

المرحلة الأولى: الإصلاح الطارئ

1. هامش Paper Trading لا يُخصم فعلياً حرج
2. Alpaca يعمل دائماً في وضع Paper حرج
3. أوامر Limit الورقية لا تُنفذ أبداً حرج
4. تسجيل BullMQ مكرر يسبب انهيار حرج
5. تخزين مفاتيح API مشفرة في الذاكرة المؤقتة حرج
6. فحوصات مخاطر مزدوجة في V1 عالي
7. تباعد سلوكي بين V1 و V2 عالي
8. ExchangeSync يجلب صفقات جميع المستخدمين عالي

المرحلة الثانية: تصحيح البيانات

9. خطأ تسجيل سعر BTC بقيمة \$1,921.80 حرج
10. صفقة ETH عالقة لمدة 131 ساعة حرج
11. صفقات DOGE و SOL المتكررة كل 8-10 ثوانٍ حرج
12. نمط إغلاق "يدوي" بعد 4 ساعات بالضبط عالي

المرحلة الثالثة: التحسين الخوارزمي

13. إعادة تصميم نسبة المخاطرة/المكافأة عالي
14. معالجة التحيز في اتجاه البيع عالي
15. تحسين أحجام المراكز لمنفذ Agent متوسط
16. تطوير آلية إغلاق ذكية عالي
17. إضافة فلتر تكرار الصفقات حرج

المرحلة الرابعة: التطوير الهيكلي

18. توحيد مساري V1 و V2 عالي
19. تقسيم AIOrchestrator العملاق متوسط

متوسط	20. توحيد المؤشرات المكررة
متوسط	21. تطوير نظام رصد الأداء الفوري
الملحق	
8 أسابيع	الجدول الزمني التنفيذي
تقييم	مصفوفة الأولويات والأثر المالي
2	Roua Trading - الخطة التنفيذية للإصلاح

14

يوم عمل متوقع

5

ملفات تحتاج تعديل

8

أخطاء حرجة وعالية

تحذير: تأثير مالي مباشر

المشاكل في هذه المرحلة تسبب خسائر مالية حقيقية أو مخاطر أمنية حرجة. يجب إيقاف التداول الآلي حتى يتم إصلاح البنود 1 و 3 و 17 كحد أدنى.

حرج

1. هامش Paper Trading لا يُخصم فعلياً

المشكلة: عند فتح صفقة في وضع Paper Trading، لا يتم خصم الهامش من رصيد المحفظة. الرصيد يبقى ثابتاً بغض النظر عن عدد الصفقات المفتوحة، مما يسمح بفتح عدد غير محدود من الصفقات.

الملف: `apps/api/src/modules/execution/adapters/paper-trading.adapter.ts`

الأثر: **خسارة محتملة** لا حدود للمخاطر المالية في بيئة الاختبار، مما يعني أن الاستراتيجيات قد تعمل في Paper ولكنها ستفشل فوراً في البيئة الحية.

الإصلاح المطلوب:

- 1 إضافة نظام تتبع الهامش في `PaperTradingAdapter` : خصم قيمة الصفقة من الرصيد عند الفتح وإضافتها عند الإغلاق
- 2 تطبيق فحص الهامش قبل فتح أي صفقة جديدة: إذا كان الهامش المتاح غير كافٍ، رفض الصفقة
- 3 إضافة حقول `usedMargin` و `availableBalance` في نموذج Paper Wallet
- 4 كتابة اختبارات وحدة للتحقق من خصم الهامش ورفض الصفقات عند عدم كفاية الرصيد

2. Alpaca يعمل دائماً في وضع Paper

المشكلة: محول Alpaca لا يحتوي على مسار كود للتداول الحي. جميع الطلبات تمر عبر `paperURL` بغض النظر عن إعدادات المستخدم، مما يجعل التداول الحي مستحيلًا.

الملف: `apps/api/src/modules/execution/adapters/alpaca.adapter.ts`

الأثر: **عائق حرج** المنصة لا يمكنها تنفيذ صفقات حقيقية على الإطلاق عبر Alpaca.

الإصلاح المطلوب:

- 1 إضافة منطق تبديل البيئة: إذا كان `isPaperMode = false`، استخدم `liveURL` بدلاً من `paperURL`
- 2 إضافة فحص أمان: تأكيد مزدوج قبل التبديل للحي + سجل تدقيق لكل طلب حي
- 3 إضافة متغير بيئة `ALPACA_LIVE_ENABLED` مع قيمة افتراضية `false`
- 4 اختبار شامل للتداول الحي بمبالغ صغيرة جداً قبل التفعيل الكامل

3. أوامر Limit الورقية لا تُنفذ أبداً

المشكلة: في Paper Trading، أوامر Limit تبقى معلقة إلى الأبد. لا توجد آلية للتحقق الدوري من سعر السوق مقابل سعر ال Limit لتنفيذ الأمر. أوامر Market فقط هي التي تُنفذ.

الملف: `apps/api/src/modules/execution/adapters/paper-trading.adapter.ts`

الأثر: **خسارة فرص** جميع استراتيجيات Limit Order لا يمكن اختبارها. هذا يشمل أوامر الشراء عند الانخفاض وأوامر جني الأرباح.

الإصلاح المطلوب:

- 1 إضافة `CronJob` يعمل كل 5-10 ثوانٍ: يفحص جميع أوامر Limit المعلقة ويقارن سعر السوق الحالي بسعر ال Limit
- 2 تنفيذ الأمر تلقائياً إذا وصل سعر السوق لسعر ال Limit: شراء إذا كان سعر السوق أقل أو يساوي Limit، بيع إذا كان أعلى أو يساوي
- 3 إضافة حد أقصى لمدة بقاء أمر Limit المعلق (مثلاً 24 ساعة) مع إلغاء تلقائي بعد انتهاء المدة
- 4 إرسال إشعار عند تنفيذ أو إلغاء أمر Limit

حرج

4. تسجيل BullMQ مكرر يسبب انهيار

المشكلة: مسار V2 يسجل معالجات BullMQ في كل مرة يتم فيها إنشاء مثيل جديد. إذا تم إعادة تنشيط الوحدة، يتم تسجيل معالجات مكررة، مما يسبب تنفيذ كل أمر مرتين وانهيار النظام.

الملف: `apps/api/src/trading/order-state-manager.ts` و `order-dispatcher.ts`

الأثر: **خطر انهيار** كل طلب يُنفذ مرتين، مما يضاعف المخاطر ويتسبب في أخطاء في الرصيد.

الإصلاح المطلوب:

- 1 إضافة حماية `singleton` على تسجيل المعالجات: استخدام `Set` لتتبع المعالجات المسجلة ورفض التسجيل المكرر
- 2 تطبيق `OnModuleDestroy` لتنظيف المعالجات عند إيقاف الوحدة
- 3 إضافة فحص عند بدء التشغيل: إذا كانت المعالجات مسجلة مسبقاً، تخطي التسجيل
- 4 كتابة اختبار تكاملي يحاكي إعادة تنشيط الوحدة ويتحقق من عدم تكرار التنفيذ

حرج

5. تخزين مفاتيح API مشفرة في الذاكرة المؤقتة

المشكلة: `AdapterCacheService` يخزن مفاتيح API المشفرة في ذاكرة مؤقتة. في حالة اختراق الذاكرة أو تسريبها، يمكن استخراج المفاتيح وفك تشفيرها.

الملف: `apps/api/src/modules/execution/adapters/cache.service.ts`

الأثر: **خطر أمني** مفاتيح API للتبادلات الحية (Binance, Alpaca) معرضة للاختراق.

الإصلاح المطلوب:

- 1 عدم تخزين مفاتيح API في الذاكرة المؤقتة مطلقاً؛ جلبها مباشرة من قاعدة البيانات عند الحاجة فقط
- 2 إذا كان الأداء يتطلب التخزين المؤقت: استخدام `Vault` أو `AWS Secrets Manager` بدلاً من التخزين المحلي
- 3 تطبيق مبدأ الصلاحيات الدنيا: المفاتيح تبقى في الذاكرة لأقل مدة ممكنة ثم تُحذف فوراً
- 4 إضافة تشفير إضافي في الذاكرة باستخدام `crypto.createCipheriv` مع مفتاح مشتق من متغير بيئة

عالي

6. فحوصات مخاطر مزدوجة في V1

المشكلة: في مسار V1، يتم تنفيذ فحص المخاطر مرتين: مرة في `RiskGatekeeper` ومرة في `RiskManager` داخل `TradingService`. هذا يسبب تأخيراً غير ضروري واحتمال تناقض في النتائج.

الملف: `apps/api/src/trading/trading.controller.ts` و `trading.service.ts`

الأثر: **كفاءة** تأخير مزدوج في التنفيذ وإمكانية رفض صفقات مقبولة أو قبول صفقات خطيرة.

الإصلاح المطلوب:

إزالة فحص المخاطر المكرر من `TradingService` والاعتماد حصرياً على `RiskGatekeeper` كبوابة وحيدة. توحيد منطق المخاطر في مكان واحد.

المشكلة: مسار (`TradingController` → `TradingService`) V1 ومسار (`OrderController`) V2 (`IdempotencyService` → `OrderStateManager` → `RabbitMQ`) لهما سلوك مختلف تماماً في معالجة الأخطاء وفحص المخاطر وتأكيد الطلبات.

الأثر: تناقض نفس الطلب قد يُقبل في V1 ويُرفض في V2 أو العكس.

الإصلاح المطلوب:

توحيد المسارين في V2 تدريجياً: V1 يصبح واجهة توافقية تستدعي V2 داخلياً. انظر المرحلة الرابعة للمزيد من التفاصيل.

8. ExchangeSync يجلب صفقات جميع المستخدمين

المشكلة: خدمة `ExchangeSync` تجلب جميع الصفقات من التبادلات بدون تصفية حسب المستخدم، مما قد يخلط بيانات مستخدم بآخر.

الملف: `apps/api/src/trading/exchange-sync.service.ts`

الأثر: بيانات مزج صفقات مستخدمين مختلفين وإحصائيات خاطئة.

الإصلاح المطلوب:

إضافة تصفية `userId` في جميع استعلامات المزامنة. كل مستخدم يرى صفقاته فقط.

2 تصحيح شذوذ البيانات

من خلال التحليل المعمق لأكثر من 300 صفقة حقيقية مسجلة بين 22 مايو و 8 يونيو 2026، تم اكتشاف عدة شذوذ حرجة في البيانات تشير إلى أخطاء برمجية في تسجيل البيانات وتنفيذ الصفقات. هذه الأخطاء تؤثر مباشرة على دقة الإحصائيات والمخاطر المالية.

حرج

9. خطأ تسجيل سعر BTC بقيمة \$1,921.80

المشكلة: في 4 يونيو الساعة 3:25 صباحاً، تم تسجيل صفقة BTC بسعر دخول كانحوالي $11,921.80$ بينما السعر الحقيقي 921.80 ، بينما السعر الحقيقي $63,000$. هذا أدى إلى حساب خسارة وهمية بقيمة -177.47 .

السبب المحتمل: خطأ في تحويل وحدات الأسعار من الساتوشي إلى الدولار، أو خطأ في تنسيق البيانات من Binance API (ربما تم قراءة حجم العملة كسعر).

الأثر: بيانات فاسدة الإحصائيات والتقارير المالية لا تعكس الواقع.

الإصلاح المطلوب:

1 فحص `BinanceAdapter` و `MarketDataAggregator` : التأكد من أن الأسعار تُقرأ من الحقل الصحيح في استجابة API

2 إضافة `Price Validation Layer` : فحص السعر المسجل مقابل نطاق معقول (مثلاً: BTC بين $50,000 - 100,000$). إذا كان خارج النطاق، رفضه وطلب إعادة الجلب

3 تنظيف قاعدة البيانات: تصحيح أو حذف السجلات ذات الأسعار غير المنطقية

4 إضافة سجل تدقيق لكل تسجيل سعر مع المصدر والطابع الزمني

10. صفقة ETH عالقة لمدة 131 ساعة

حرج

المشكلة: صفقة ETH ظلت مفتوحة لمدة 131 ساعة و 12 دقيقة مع P&L مسجل بقيمة \$0.00. هذا يشير إلى صفقة "شبح" لم تُغلق بشكل صحيح.

السبب المحتمل: فشل في آلية الإغلاق التلقائي أو خطأ في مزامنة حالة الصفقة مع التبادل.

الأثر: رأس مال محجوز رأس المال محجوز في صفقة لا تعمل والمخاطر مفتوحة بلا سيطرة.

الإصلاح المطلوب:

- 1 إضافة **Position Health Check** : فحص دوري كل 15 دقيقة لحالة جميع الصفقات المفتوحة
- 2 تعيين حد أقصى لمدة الصفقة: إذا تجاوزت 24 ساعة بدون تحديث P&L، إرسال تنبيه. إذا تجاوزت 48 ساعة، إغلاق تلقائي
- 3 إضافة آلية **Stale Position Detector** : تحديد الصفقات التي لم يتغير P&L لها لمدة تزيد عن ساعة واحدة
- 4 تصحيح الصفقات العالقة في قاعدة البيانات: إغلاقها يدوياً بالسعر الحالي

11. صفقات DOGE و SOL المتكررة كل 10-8 ثوانٍ

حرج

المشكلة: منفذ Agent يفتح صفقات متطابقة تقريباً على DOGE و SOL كل 10-8 ثوانٍ. هذا يشير إلى خطأ في منطق **SmartExecutor** الذي يعمل كل 10 ثوانٍ ويعيد قراءة نفس التوصية من Trading Briefs دون التحقق من وجود صفقة مفتوحة بالفعل.

الملف: `apps/api/src/modules/ai/smart-executor.ts`

الأثر: خسارة مالية تراكم صفقات مكررة يضاعف المخاطر ويتسبب في خسائر متسلسلة عند الوصول لوقف الخسارة.

الإصلاح المطلوب:

- 1 إضافة **Duplicate Trade Filter** في SmartExecutor: قبل فتح صفقة جديدة، فحص ما إذا كانت هناك صفقة مفتوحة على نفس الأصل بنفس الاتجاه خلال آخر 5 دقائق
- 2 إضافة **Cooldown Period** : بعد فتح صفقة على أصل معين، انتظار 30 دقيقة على الأقل قبل فتح صفقة أخرى على نفس الأصل
- 3 تسجيل معرف التوصية (Brief ID) مع الصفقة: عدم تنفيذ نفس التوصية مرتين
- 4 إضافة حد أقصى للصفقات المفتوحة في نفس الوقت: مثلاً 3 صفقات كحد أقصى لكل أصل و 10 صفقات إجمالية

12. نمط إغلاق "يدوي" بعد 4 ساعات بالضبط

عالي

المشكلة: العديد من الصفقات تُغلق بسبب "Manual" بعد 4 ساعات بالضبط من الفتح. هذا ليس إغلاقاً يدوياً حقيقياً بل هو مهلة زمنية تلقائية (**timeout**) مُسمّاة بشكل خاطئ.

الأثر: إحصائيات خاطئة تضليل في تقارير الأداء وإحصائيات أسباب الإغلاق.

الإصلاح المطلوب:

- 1 إضافة سبب إغلاق جديد: **TIMEOUT** منفصل عن **MANUAL**
- 2 تحديث جميع سجلات الإغلاق الحالية: تغيير "Manual" إلى "Timeout" للصفقات التي أُغلقت بعد 4 ساعات بالضبط
- 3 إضافة خيار تهيئة لمدة المهلة الزمنية: السماح للمستخدم بتعديل فترة الـ Timeout حسب الأصل والاستراتيجية

3 التحسينات الخوارزمية

التحليل الكمي لبيانات التداول يكشف عن مشاكل هيكلية في آليات اتخاذ القرار. هذه المشاكل ليست أخطاء برمجية بل نقاط ضعف في التصميم الخوارزمي تحتاج إلى إعادة تصميم ذكية.

3%

نسبة الوصول لـ TP

5:1

نسبة SL أقرب من TP

+85%

صفقات بيع (تحيز اتجاهي)

عالي

13. إعادة تصميم نسبة المخاطرة/المكافأة

المشكلة: وقف الخسارة (SL) موضوع أقرب بكثير من جني الأرباح (TP) من نقطة الدخول. النسبة الحالية تقريباً 1:5 (SL:TP)، مما يعني أن السعر يحتاج للتحرك 5 أضعاف المسافة للوصول لـ TP مقارنة بـ SL. النتيجة: معظم الصفقات تضرب SL قبل الوصول لـ TP.

البيانات: من 300+ صفقة، أقل من 3% وصلت لـ TP. أكثر من 60% ضربت SL. الباقي أُغلق بالـ Timeout بخسارة أو بدون ربح.

الإصلاح المطلوب:

- 1 تعديل النسبة إلى 1:2 على الأقل (SL:TP): إذا كان SL عند -2%، يجب أن يكون TP عند +4% كحد أدنى
- 2 استخدام **ATR-Based SL/TP**: حساب SL و TP بناءً على متوسط المدى الحقيقي (ATR) بدلاً من نسب ثابتة
- 3 تطبيق **Trailing Stop**: تحريك SL تدريجياً مع حركة السعر لصالح الصفقة، لحماية الأرباح المكتسبة
- 4 إضافة **Partial TP**: إغلاق 50% من الصفقة عند TP1 والباقي عند TP2 مع تحريك SL لنقطة الدخول

المشكلة: أكثر من 85% من الصفقات هي صفقات بيع (SELL/SHORT). هذا التحيز الشديد يعني أن النظام لا يستفيد من حركات السوق الصاعدة، ويخسر في الأسواق الصاعدة بشكل مستمر.

السبب المحتمل: **StrategicCouncil** يميل لتوليد توصيات بيع، أو **SignalGenerator** لديه تحيز في المؤشرات الفنية.

الإصلاح المطلوب:

- 1 تحليل سجلات **StrategicCouncil** : هل نسبة التوصيات بيع أيضاً أم أن التحيز يأتي من SmartExecutor؟
- 2 إضافة **Direction Bias Monitor** : تنبيه تلقائي عندما يتجاوز التحيز 70% في أي اتجاه
- 3 إضافة **Counter-Trend Filter** : فحص الاتجاه العام للسوق قبل تنفيذ التوصيات. في سوق صاعد، تقليل صفقات البيع
- 4 تطبيق **Balanced Execution** : حد أقصى للصفقات في اتجاه واحد (مثلاً 70% كحد أقصى للبيع)

المشكلة: منفذ Agent يفتح صفقات بأحجام صغيرة جداً (يفتح صفقات كبيرة Smart منفذ. بينما الخسائر والعمولات تتأكل معظم الأرباح) $(200 - 150150 - 200)$ بينما الخسائر والعمولات تتأكل معظم الأرباح. منفذ Smart يفتح صفقات كبيرة $(500 - 5000 +)$ مما يضعف المخاطر. لا يوجد توازن في إدارة رأس المال.

الإصلاح المطلوب:

- 1 تطبيق **Kelly Criterion** لحساب حجم المركز الأمثل بناءً على نسبة الفوز ومتوسط الربح/الخسارة
- 2 تعيين نسبة ثابتة من رأس المال لكل صفقة: مثلاً 1-2% من إجمالي رأس المال كحد أقصى للمخاطرة
- 3 تعديل أحجام Agent: زيادة تدريجية بناءً على الأداء. البدء بـ والتدرج حتى 300300 والتدرج حتى 1000
- 4 تعديل أحجام Smart: تقليص تدريجي وإضافة حد أقصى مطلق (مثلاً 5% من رأس المال)

المشكلة: معظم الصفقات تُغلق إما بـ SL أو بال Timeout بعد 4 ساعات. لا توجد آلية إغلاق ذكية تأخذ في الاعتبار ديناميكيات السوق الحالية.

الإصلاح المطلوب:

- 1 تطبيق **Dynamic Exit Strategy** : الإغلاق بناءً على مؤشرات فنية (RSI, MACD crossover) بدلاً من الوقت فقط
- 2 إضافة **Breakeven Move** : تحريك SL لنقطة الدخول عندما يصل الربح لـ $1.5 \times$ مسافة SL
- 3 تطبيق **Time-Based Decay** : تقليل TP تدريجياً مع مرور الوقت لضمان عدم تفويت الأرباح
- 4 إضافة **Momentum Exit** : إغلاق الصفقة عند فقدان الزخم (مثلاً RSI ينعكس من فوق 70 لأقل من 60 في صفقة شراء)

المشكلة: هذا هو السبب الجذري للمشكلة رقم 11. `SmartExecutor` يعمل كل 10 ثوانٍ ويقرأ نفس التوصية وينفذها مرة أخرى. لا يوجد أي فلتر للتحقق من وجود صفقة نشطة على نفس الأصل.

الملف: `apps/api/src/modules/ai/smart-executor.ts` - الدالة `handleTick()`

الإصلاح المطلوب:

- 1 في بداية `handleTick()` : استعلام قاعدة البيانات عن الصفقات المفتوحة حالياً على الأصل المستهدف
- 2 إضافة `Execution Fingerprint` : تجزئة (symbol + direction + briefId) لكل تنفيذ. إذا كان التوقيع موجوداً في آخر 30 دقيقة، تخطي التنفيذ
- 3 إضافة `Rate Limiter` : حد أقصى صفقة واحدة لكل أصل كل 30 دقيقة لـ Agent، وكل 60 دقيقة لـ Smart
- 4 إضافة سجل `ExecutionLog` : تسجيل كل محاولة تنفيذ مع السبب (نفذ / رفض / تجاوز حد)

4 التطويرات الهيكلية

عالي

18. توحيد مساري V1 و V2

المشكلة: وجود مسارين مختلفين لمعالجة الطلبات يسبب تناقضات في السلوك وصعوبة في الصيانة. V1 مبني على `TradingService` المباشر، و V2 يستخدم `RabbitMQ` و `IdempotencyService` و `OrderStateManager`.

خطة التوحيد:

- 1 الأسبوع 5: تحويل `TradingController` (V1) ليصبح واجهة توافقية تستدعي `OrderController` (V2) داخلياً
- 2 الأسبوع 6: نقل جميع العملاء إلى V1 API. V2 يبقى للتوافق مع الإشارات القديمة
- 3 الأسبوع 7: إزالة `TradingService` القديم بالكامل ونقل المنطق المتبقي إلى V2
- 4 الأسبوع 8: اختبارات تكاملية شاملة ومراقبة الأداء

متوسط

19. تقسيم AIOrchestrator العملاق

المشكلة: `AIOrchestrator` ملف عملاق يتجاوز 1000 سطر ويحتوي على 11 مزود AI مع سلاسل احتياطية. هذا يجعل الصيانة والإضافة والاختبار صعباً جداً.

الملف: `apps/api/src/modules/ai/ai-orchestrator.service.ts`

الإصلاح المطلوب:

- 1 استخراج كل مزود AI إلى خدمة مستقلة ضمن مجلد `/providers`
- 2 إنشاء `ProviderRegistry`: سجل مركزي للمزودين مع واجهة موحدة
- 3 استخراج منطق سلاسل الاحتياط إلى `FallbackChainManager` مستقل
- 4 تقليل AIOrchestrator لدور منسق فقط: اختيار المزود + إدارة الفشل + تسجيل النتائج

المشكلة: محرك `ScannerService` (2100+ سطر) يحسب مؤشرات فنية مكررة موجودة بالفعل في `TechnicalIndicatorService`. هذا يضيع الموارد ويسبب نتائج مختلفة لنفس المؤشر.

الإصلاح المطلوب:

- 1 تحديد جميع المؤشرات المكررة بين `ScannerService` و `TechnicalIndicatorService`
- 2 توحيد الحسابات في `TechnicalIndicatorService` كمصدر وحيد
- 3 تحديث `ScannerService` لاستدعاء `TechnicalIndicatorService` بدلاً من الحساب المحلي
- 4 إزالة كود Ichimoku الميت من `ScannerService`
- 5 نقل `PerformanceTracker` من وحدة AI إلى وحدة Analytics

المشكلة: لا توجد لوحة مراقبة فورية للأداء الخوارزمي. الأخطاء والشذوذ في البيانات لا تُكتشف إلا بعد مراجعة يدوية، مما يتأخر اكتشاف المشاكل لأسابيع.

الإصلاح المطلوب:

- 1 إنشاء `PerformanceMonitor` : خدمة مراقبة فورية تحسب Win Rate و Profit Factor و Max Drawdown في الوقت الحقيقي
- 2 إضافة `Anomaly Detector` : تنبيه تلقائي عند اكتشاف شذوذ (مثل: 5 صفقات متتالية خاسرة، أو تحيز اتجاهي يزيد عن 70%)
- 3 إنشاء `Circuit Breaker` : إيقاف التداول الآلي تلقائياً عند تجاوز حدود الخسارة اليومية (مثلاً: خسارة 5% من رأس المال)
- 4 لوحة مراقبة على الويب: عرض الإحصائيات والتنبيهات في الوقت الحقيقي

الجدول الزمني ومصفوفة الأولويات

ج

الجدول الزمني التنفيذي

الأسبوع	المرحلة	المهام	المخرجات
1	طارئ	إصلاح #1 (هامش Paper)، #17، #3 (Limit Orders) (فلتر التكرار)	إيقاف نزيف الصفقات المكررة وتفعيل Limit Orders
2	طارئ	إصلاح #2 (Alpaca Live)، #5، #4 (BullMQ) (أمان API)، #6 (مخاطر مزدوجة)	تفعيل التداول الحي وإصلاح الأمان
2-3	تصحيح	إصلاح #9 (سعر BTC)، #10 (صفقة عالقة)، #11 (صفقات مكررة)، #12 (سبب الإغلاق)	بيانات نظيفة وإحصائيات دقيقة
3-4	تحسين	إصلاح #13 (SL/TP)، #14 (تحيز البيع)، #16 (إغلاق ذكي)	تحسين نسبة الفوز وتقليل الخسائر
4-5	تحسين	إصلاح #15 (أحجام المراكز)، اختبارات شاملة للخوارزميات	إدارة مخاطر متوازنة
5-7	هيكلي	إصلاح #18 (توحيد V1/V2)، #20، #19 (AIOrchestrator) (مؤشرات مكررة)	بنية نظيفة قابلة للتطوير
7-8	هيكلي	إصلاح #21 (رصد الأداء)، اختبارات نهائية، مراقبة الإنتاج	نظام مراقبة ورصد فوري

مصفوفة الأولويات والأثر المالي

أولوية قصوى - إيقاف الخسائر

- #17 فلتر تكرار الصفقات - يوقف النزيف الفوري
- #1 هامش Paper Trading - يحجب المخاطر غير المحدودة
- #3 Limit Orders لا تُنفذ - يعطل استراتيجيات كاملة
- #4 BullMQ مكرر - يضاعف كل صفقة
- #9 سعر BTC خاطئ - يفسد الإحصائيات

أولوية عالية - تحسين الربحية

- #13 نسبة SL/TP - تؤثر على 97% من الصفقات
- #14 تحيز البيع - يفوت 50% من الفرص
- #16 إغلاق ذكي - يحسن نقاط الخروج
- #2 Alpaca حي - يفتح التداول الحقيقي
- #18 توحيد V1/V2 - يزيل التناقضات

أولوية متوسطة - تحسين الكفاءة

- #10 صفقة ETH عالقة - يحسن إدارة رأس المال
- #12 سبب الإغلاق - يحسن الإحصائيات
- #15 أحجام المراكز - يحسن إدارة المخاطر
- #7 توحيد V1/V2 سلوكياً

أولوية تحسينية - تطوير طويل الأمد

- #5 أمان مفاتيح API - حماية طويلة الأمد
- #6 فحص مخاطر مزدوج - كفاءة التنفيذ
- #8 تصفية ExchangeSync - سلامة البيانات
- #19 تقسيم AIOrchestrator - قابلية الصيانة

